Übungsblatt 9: C, Assembler und Lüfterregler

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Die Werkzeugkette prüfen (10 Punkte)

1. Ein RISC-V-GNU-Zieltreiber wird mit `-S lab_09_transfer.c -o transfer.o` aufgerufen. Welche Art Inhalt entsteht trotz der Endung .o? Begründen Sie. (2 P)
2. Geben Sie passende Aufrufe für Assemblerausgabe und anschließend Objektdatei mit `-march=rv32i -mabi=ilp32` an. Ordnen Sie Compiler und Assembler zu. (3 P)
3. Ein Header deklariert `extern int helper(int);`, die Funktion wird aufgerufen, aber nirgends definiert. Welche Stufe kann am fehlenden Funktionskörper scheitern? Warum behebt eine weitere Deklaration das nicht? (2 P)
4. Erläutern Sie march und mabi. Warum genügt ein Host-gcc ohne passende Zielauswahl nicht? (3 P)

Die neue C-Datei steht unter `beispiele/lab_09_transfer.c`. Die Aufrufe sind zu formulieren; eine vorhandene Cross-Toolchain ist für diese Papieraufgabe nicht nötig.

Musterlösung

1. Assemblertext: -S bestimmt die letzte Stufe; die frei gewählte Ausgabedateiendung ändert den Inhalt nicht. Inhalt und Begründung je 1 P.
2. Beispielsweise mit passendem GNU-Zieltreiber:

```
riscv64-unknown-elf-gcc \
  -march=rv32i -mabi=ilp32 -S \
  beispiele/lab_09_transfer.c -o transfer.s
riscv64-unknown-elf-gcc \
  -march=rv32i -mabi=ilp32 -c \
  transfer.s -o transfer.o
```

Erster Aufruf endet mit Compiler-Assemblerausgabe, zweiter assembliert zur Objektdatei. Aufrufe je 1 P, Stufenzuordnung 1 P. Dies sind keine behaupteten Ausführungsprotokolle. 3. Der Linker kann das benötigte Symbol nicht auflösen (1 P). Eine Deklaration liefert keinen implementierten Maschinenkörper (1 P). 4. march wählt ISA/Erweiterungen, mabi die Binärschnittstelle; ilp32 verwendet hier 32-Bit-int, long und Zeiger. Ein Hostcompiler kann eine ganz andere Zielarchitektur erzeugen. Je 1 P.

Aufgabe 2: Einen neuen Kontrollfluss übersetzen (12 Punkte)

1. Das Fragment soll für negatives x dessen Betrag, sonst x + 2 zurückgeben. Eingaben liegen zwischen −100 und 100. Prüfen Sie x = −4 und x = 5 und reparieren Sie den Fehler. (4 P)

```
    blt a0, x0, Negativ
    addi a0, a0, 2
Negativ:
    sub a0, x0, a0
    ret
```

2. Übersetzen Sie folgende C-Funktion vollständig nach RV32I mit Ergebnis in a0. Bestimmen Sie außerdem Körper- und Testanzahl für x = 7 sowie das Ergebnis für x = −1. (6 P)

```
int lower_to_zero(int x) {
    while (x > 0) { x -= 3; }
    return x;
}
```

3. a0 enthält `0xFFFFFFFA`. Welche signed und unsigned Werte sind das? Weshalb wäre ein unsigned Schleifentest hier falsch? (2 P)

Musterlösung

1. −4 wird korrekt 4. Für 5 entsteht erst 7, dann fälschlich −7. Nach der Erhöhung fehlt `jal x0, Ende` mit `Ende: ret` hinter dem Negativzweig. Fälle je 1 P, fehlende Kante und Korrektur zusammen 2 P.
2. Eine manuelle Lösung:

```
lower_to_zero:
Test:
    bge x0, a0, Ende
    addi a0, a0, -3
    jal x0, Test
Ende:
    ret
```

Für 7: Werte 7 → 4 → 1 → −2, drei Körper, vier Tests. Für −1: kein Körper, Ergebnis −1. Code 4 P (Test, Update, Rückkante, Rückkehr je 1); Trace und Randfall je 1 P. 3. Signed −6, unsigned 4294967290 (zusammen 1 P). Negative signed Eingaben sollen sofort enden, werden unsigned aber als große positive Zahlen gelesen (1 P).

Aufgabe 3: Jedes zweite Element summieren (12 Punkte)

ILP32: int und Zeiger je 32 Bit. Gültiges Array $[6, -2, 9, 4, -7, 3]$ ab `B=0x5600`. Alle betrachteten signed Zwischensummen sind darstellbar.

1. Geben Sie Adresse und Wert von p[4] an. Ist `0x5618` ein vorhandenes Element? (4 P)
2. Die Blattfunktion addiert p[0], p[2], p[4], soweit der Index kleiner als n ist. a0 enthält B, a1 die unsigned Länge, $0 \leq n \leq 6$. Füllen Sie vier Befehlsnamen und die letzte Konstante ein. (5 P)

```
sum_even_positions:
    mv t0, a0
    li t1, 0
    li t2, 0
Loop:
    _______ t1, a1, Ende
    _______ t3, t1, 2
    _______ t4, t0, t3
    _______ t5, 0(t4)
    add t2, t2, t5
    addi t1, t1, _______
    jal x0, Loop
Ende:
    mv a0, t2
    ret
```

3. Bestimmen Sie Ergebnis und Loadzahl für n = 0 und n = 6. Welche Grenze des C-Vergleichs wäre bei einer anderen signed Summe zu beachten? (3 P)

Musterlösung

1. `0x5600`$+4\cdot 4 =$ `0x5610`, Wert −7. Je 1 P. `0x5618` liegt unmittelbar hinter sechs Elementen und darf nicht als p[6] gelesen werden (2 P).
2. bgeu, slli, add, lw und Konstante 2. Je 1 P. Die neue vollständige Funktionsquelle steht in `beispiele/lab_09_alternate_sum.s`.
3. n = 0: 0 und kein Load; n = 6: $6 + 9 - 7 = 8$ und drei Loads (zusammen 2 P). Signed C-Überlauf wäre undefiniert; die RV32I-Moduloaddition kann ihn nicht legitimieren (1 P).

Aufgabe 4: Spiegeln und verschieben (8 Punkte)

`Negate` ist der Leaf `sub a0, x0, a0`, danach ret. Eingaben liegen zwischen −100 und 100. Normale ILP32-Aufrufe, Eintritt sp = `0xC000`.

```
reflect_plus_four:
    addi sp, sp, -16
    sw ra, 4(sp)
    jal ra, Negate
    addi a0, a0, 4
    lw ra, 4(sp)
    addi sp, sp, 16
    ret
```

1. Nennen Sie reservierten sp, Sicherungsadresse und sp nach Rückkehr. (3 P)
2. Begründen Sie ra-Sicherung und Framegröße. (3 P)
3. Berechnen Sie Ergebnisse für a0 = 6 und a0 = −3. (2 P)

Musterlösung

1. sp = `0xBFF0`, ra-Slot `0xBFF4`, nachher sp = `0xC000`. Je 1 P.
2. Der innere jal überschreibt ra, dessen alter Wert für die äußere Rückkehr nötig bleibt (2 P). Ein Wort benötigt vier Byte Nutzbedarf, aber die vereinbarte Aufrufausrichtung verlangt hier 16 Byte Frame (1 P).
3. $-6+4=-2$, $-(-3)+4=7$. Je 1 P. Die Rückgabe liegt in a0; ein alter a0-Wert darf sie nicht nachträglich überschreiben.

Aufgabe 5: Ein dreistufiger Lüfterregler (10 Punkte)

Zustände: 0 aus, 1 langsam, 2 schnell. Ereignisse: 0 keines, 1 PLUS (UP), 2 MINUS (DOWN), 3 STOP (LEFT). PLUS erhöht bis höchstens 2, MINUS vermindert bis mindestens 0, STOP setzt null. Kein Ereignis hält den Zustand; ungültige Zustandswerte werden immer null.

Rohmasken: UP = 1, DOWN = 2, LEFT = 4, RIGHT = 8. Null gibt frei; die nächste nichtleere Gruppe verbraucht die Freigabe. Nur eine einzelne UP-, DOWN- oder LEFT-Taste liefert ein Ereignis. RIGHT und Mehrfachdruck liefern 0, verbrauchen aber ebenfalls die Freigabe. Anfang: Zustand 0, Freigabe geschlossen.

1. Erstellen Sie die Übergangstabelle für alle gültigen Zustände und Ereignisse. (4 P)
2. Tragen Sie zu jeder Rohmaske Ereignis und Zustand danach ein. (4 P)

Rohmaske	0	1	1	0	1	0	2	0	3	2	0	4
Ereignis												
Zustand												

3. Zustandswert 17 und Ereignis 0: Welche Rückgabe ist nötig, und welcher zusätzliche Schritt speichert die Korrektur tatsächlich? (2 P)

Musterlösung

Zustand	0 keines	1 PLUS	2 MINUS	3 STOP
0	0	1	0	0
1	1	2	0	0
2	2	2	1	0

1. Je Tabellenzelle 1/3 P.

Rohmaske	0	1	1	0	1	0	2	0	3	2	0	4
Ereignis	0	1	0	0	1	0	2	0	0	0	0	3
Zustand	0	1	1	1	2	2	1	1	1	1	1	0

2. Je Zelle 1/6 P. Die Mehrfachmaske drei unterdrückt auch die danach verbleibende Taste zwei, bis wieder null gelesen wurde.
3. Rückgabe null; der Caller muss sie ins Zustandsregister kopieren, beispielsweise `mv s0, a0`. Je 1 P. Eine Marke allein ändert keinen gespeicherten Zustand.

Aufgabe 6: Gleiche ISA-Wirkung, anderer Zeitablauf (8 Punkte)

1. Im addi-erweiterten Einzyklusmodell: Nennen Sie RegWrite, ALUSrc und MemWrite für addi. Warum muss MemtoReg die ALU wählen? (3 P)
2. Zeichnen Sie das Pipeline-Raster des folgenden Kerns bis zum letzten WB. IF/ID/EX/MEM/WB, Forwarding aus EX/MEM und MEM/WB, anfangs leer, Loadwert erst nach MEM; keine anderen Konflikte. a0 zeigt auf ein Wort −6. Nennen Sie Stallzahl, Quellen beider Subtraktionsoperanden, Endwert t1 und Gesamttakte. (5 P)

```
lw   t0, 0(a0)
addi t2, x0, 4
sub  t1, t2, t0
```

Musterlösung

1. RegWrite = 1, ALUSrc = 1 (zusammen 1 P), MemWrite = 0 (1 P). Das Resultat entsteht in der ALU und nicht im Datenspeicher, deshalb MemtoReg = 0 (1 P).

Befehl / Takt	1	2	3	4	5	6	7
lw	IF	ID	EX	MEM	WB		
addi		IF	ID	EX	MEM	WB	
sub			IF	ID	EX	MEM	WB

2. Je korrekter Zeile 1 P. Kein Load-Use-Stall: Die unabhängige addi füllt die Lücke. In EX5 kommt t2 = 4 aus EX/MEM, t0 = −6 aus MEM/WB (Quellen und Stallfreiheit zusammen 1 P). t1 = 10, insgesamt sieben Takte (zusammen 1 P). Eine zusätzliche Bubble wäre hier unnötig.

Aufgabe 7: Den Lüfterregler in Ripes implementieren

1. Schreiben Sie eine reine Blattfunktion `next_level`: Zustand in a0, Ereignis in a1, Folgezustand in a0. Testen Sie alle zwölf gültigen Paare sowie Zustand 17 mit Ereignis 0.
2. Verbinden Sie Rohwerterfassung, Freigabemerker, next_level und gespeicherten Zustand. Geräte: D-Pad und mindestens ein LED-Pixel; echte Exports D_PAD_0_BASE und LED_MATRIX_0_BASE prüfen.
3. Farbe: Zustand 0 schwarz, 1 `0x000000C0`, 2 `0x00C0C0C0`. Die LED zeigt nur die gewählte Stufe; es wird kein realer Motor angesteuert.
4. Prüfen Sie zwei PLUS-Ereignisse, PLUS an der Obergrenze, MINUS, STOP, Halten, RIGHT und Mehrfachdruck mit teilweisem Loslassen. Protokollieren Sie Rohmaske, Freigabe, Ereignis, Zustand und Farbe.
5. Vergleichen Sie nach Ihrer Implementierung mit Referenzprogramm. Testen Sie auch die Rückkopie einer ungültigen Zustandskorrektur.

Lesen Sie jeden D-Pad-Wert (Wortoffsets 0, 4, 8, 12) einmal je Gruppe und prüfen Sie die Lese-Nebenwirkungen Ihrer Version. Vier Reads sind keine atomare Momentaufnahme; die Freigabe ersetzt keine physikalische Entprellung.

Musterlösung

Die neue Referenz enthält Hauptschleife und die drei Leafs read_mask, next_level und level_colour. main kehrt nicht zurück; die Helfer erhalten s-Register und sp. Die Hauptschleife speichert a0 nach next_level in s0.

Zwei getrennte PLUS-Ereignisse führen 0 → 1 → 2. Weiteres PLUS hält 2; MINUS führt nach 1, STOP nach 0. Halten zählt einmal; RIGHT erzeugt kein Ereignis und verlangt erneutes Loslassen. Mehrfachdruck verbraucht die Freigabe. Für Zustand 17 liefert und speichert die Rückkopie null und damit Schwarz. Tatsächliche Anzeigen und exportierte Adressen stammen aus dem eigenen Lauf.

Aufgabe 8: Compilervergleich mit neuen Funktionen (optional)

Übersetzen Sie `beispiele/lab_09_transfer.c` mit einer vorhandenen passenden GNU-Toolchain oder Compiler Explorer nach RV32I/ILP32, einmal -O0 und einmal -O2, jeweils mit -S.

1. Dokumentieren Sie Compiler, Version, vollständige Optionen und erzeugte Dateien.
2. Vergleichen Sie Schleifenprüfung und Indexfortschritt von lower_to_zero und sum_even_positions mit Ihrer Papierlösung.
3. Erklären Sie eine tatsächlich beobachtete Optimierung. Unterscheiden Sie gleiche Ergebnisse von gleicher Register- oder Befehlswahl.

Musterlösung

Die C-Datei verwendet eine um drei verringernde while-Schleife und eine Arrayschleife mit Indexschritt zwei. Beispiele: lower_to_zero(7) liefert –2; für $[6, -2, 9, 4, -7, 3]$ und Länge sechs liefert sum_even_positions 8. Die Arrayregion muss gültig und die signed Summe darstellbar sein.

Eine bestimmte Compilerbefehlsfolge wird ohne tatsächlichen Lauf nicht als Ergebnis zugesichert. Assemblerausgabe allein ist noch kein gelinktes, gestartetes Programm.